Labo 06 — Réponses commentées

Chaque réponse suit le même schéma : la réponse, le mécanisme, la nuance ou le piège, un exemple vérifiable au terminal.

Question 1 — « stop/start, mes données sont là »

Réponse. Il a raison sur un point : `stop` puis `start` conservent la couche writable, donc les fichiers écrits. Les données disparaissent dès que le **conteneur** disparaît : `podman rm`, `podman run --rm`, un `podman rm -f` suivi d'un nouveau `run` pour changer un paramètre ou passer à une nouvelle image, un `podman-compose down`, ou un orchestrateur qui remplace le conteneur.

Pourquoi. La couche writable appartient au conteneur, pas à l'image ni au moteur. `stop` ne détruit rien : le conteneur existe toujours (`podman ps -a`), avec sa couche. `rm` la supprime.

Nuance. La couche survit aussi à un redémarrage de la machine. Mais « ne pas supprimer le conteneur » n'est pas une stratégie : la première mise à jour d'image impose un nouveau conteneur, donc une couche vide.

Exemple.

```
podman run --name scribe alpine sh -c 'echo "note" > /data.txt'
podman start -a scribe && podman exec scribe cat /data.txt   # toujours là
podman rm scribe && podman run --rm alpine ls /data.txt      # No such file or directory
```

Question 2 — La page nginx par défaut

Réponse. `html` sans `./` n'est pas un chemin : c'est un **volume nommé** `html`, créé vide à la volée. Au premier montage sur `/usr/share/nginx/html`, qui n'est pas vide dans l'image, le moteur **copie** le contenu de l'image dans le volume (*copy-up*) — d'où la page d'accueil nginx. Votre dossier n'a jamais été monté. Correction : `-v ./html:/usr/share/nginx/html:ro`.

Pourquoi. La syntaxe `-v` distingue bind mount et volume uniquement à la forme de la source : `/`, `./`, `~` → chemin ; sinon → nom de volume. Le volume est créé sans avertissement.

Nuance. `--mount type=bind,source=html,…` aurait échoué proprement (chemin inexistant) au lieu de créer un volume en silence : c'est l'argument principal en faveur de la syntaxe longue.

Exemple.

```
podman volume ls                          # un volume "html" est apparu
podman volume inspect html --format '{{.Mountpoint}}'
podman unshare ls $(podman volume inspect html --format '{{.Mountpoint}}')   # index.html,
50x.html : copiés de l'image
podman rm -f -t 0 web && podman volume rm html
```

Question 3 — Permission denied sur le volume

Réponse. PostgreSQL tourne dans le conteneur sous l'UID 70 (`postgres`). En rootless, l'UID 70 du conteneur est traduit en un UID de votre plage `subuid` : **100069** sur une Ubuntu/WSL standard. Les fichiers appartiennent donc à 100069, et votre utilisateur 1000 n'y a aucun droit — c'est le but. `podman unshare ls -ln …` les liste, en entrant dans le user namespace de Podman.

Pourquoi. Le user namespace ne mappe que votre UID sur root ; tous les autres UID du conteneur pointent vers la plage `100000-165535`. Le noyau applique les permissions sur les UID **réels**.

Nuance. Le répertoire parent (`volumes/pgdata`) appartient bien à 1000 : seul le contenu écrit par le conteneur porte un UID traduit. Et `sudo ls` fonctionnerait aussi, mais vous feriez du root pour contourner une isolation voulue.

Exemple.

```
podman unshare cat /proc/self/uid_map
#          0       1000          1
#          1     100000      65536
podman unshare ls -ln ~/.local/share/containers/storage/volumes/pgdata/_data | head -3   # 70
70
ls -ln ~/.local/share/containers/storage/volumes/                                        #
1000 1000 pgdata
```

Question 4 — alpha ou bravo ?

Réponse. **`alpha`**. Le second conteneur trouve un `PGDATA` déjà initialisé dans le volume et ne relance pas `initdb` : `POSTGRES_PASSWORD=bravo` est ignoré, le mot de passe stocké dans la base reste `alpha`.

Pourquoi. Le script d'entrée de l'image officielle ne crée le cluster et l'utilisateur que si le répertoire est vide. Les variables `POSTGRES_*` ne servent qu'à cette initialisation.

Nuance. Pour changer le mot de passe, on passe par SQL (`ALTER USER postgres PASSWORD 'bravo'`), ou on supprime le volume — avec les données. Ce piège ressurgit en CI quand un volume traîne entre deux exécutions.

Exemple.

```
podman logs db 2>&1 | grep -c initdb     # 0 au second démarrage : rien initialisé
podman exec db psql -U postgres -c "ALTER USER postgres PASSWORD 'bravo';"
```

Question 5 — Quel type pour quel besoin

Réponse. (a) **Bind mount** : vous éditez les sources sur l'hôte, le conteneur les voit sans rebuild. (b) **Volume nommé** : données gérées par le moteur, permissions correctes, sauvegardables avec `volume export`. (c) **Bind mount en `:ro`** : un fichier versionné avec le projet, que le conteneur ne doit pas modifier. (d) **tmpfs** si la RAM le permet, sinon un bind mount ou volume dédié — surtout pas la couche writable, lente et gonflant le conteneur.

Pourquoi. Le critère est : qui doit voir et gérer les fichiers ? Vous → bind mount ; le moteur → volume ; personne après l'arrêt → tmpfs.

Nuance. Un tmpfs de 2 Go consomme 2 Go de RAM, comptés dans `--memory`. Pour (d), un volume nommé que le conteneur nettoie lui-même est souvent plus réaliste.

Exemple.

```
podman run --rm --tmpfs /tmp/pdf:size=2g alpine df -h /tmp/pdf
```

Question 6 — 1000 ne peut pas écrire chez 1000

Réponse. Les deux « 1000 » ne désignent pas le même utilisateur. Le dossier appartient à l'UID **hôte** 1000, que le conteneur voit comme **root** (`0`). L'UID 1000 **du conteneur** est, vu de l'hôte, 100999. Pour le noyau, c'est un étranger qui tente d'écrire dans votre dossier. Corrections : (1) `:U` sur le montage, le moteur fait `chown` vers l'UID du conteneur ; (2) `--userns=keep-id`, votre UID garde le numéro 1000 dans le conteneur, donc le fichier est bien à vous.

Pourquoi. `uid_map` : `0 1000 1` et `1 100000 65536`. Le 1000 du conteneur tombe dans la seconde ligne : `100000 + (1000 - 1) = 100999`.

Nuance. `:U` modifie le propriétaire **sur l'hôte**, de façon persistante (question 7). `keep-id` est fait pour les outils de développement ; il ne convient pas aux images dont le processus doit tourner sous un UID précis (postgres = 70).

Exemple.

```
podman run --rm --user 1000 -v ./sortie:/out:U alpine sh -c 'echo x > /out/f'     # OK
podman run --rm --userns=keep-id -v ./sortie:/out alpine sh -c 'id -u; echo y > /out/g'   #
1000, OK
```

Question 7 — Après `:U`

Réponse. Sur l'hôte, `ls -ln sortie` montre le dossier et les fichiers appartenant à l'UID **100999** (le 1000 du conteneur traduit), et vous ne pouvez plus y écrire sans Podman. Pour le reprendre : `podman unshare chown -R 0:0 sortie` — « root » dans le namespace, c'est vous.

Pourquoi. `:U` est un `chown` récursif réel, fait par le moteur dans le user namespace. Il n'est pas annulé au `rm` du conteneur.

Nuance. Dans le bac à sable de rédaction, la plage subuid commence à 1001 : le fichier y apparaît avec l'UID 1001. Sur votre WSL, c'est 100999. Le mécanisme est le même, seule la plage change (`cat /etc/subuid`).

Exemple.

```
ls -ln sortie            # -rw-r--r-- 1 100999 1000 … f
podman unshare chown -R 0:0 sortie && ls -ln sortie    # 1000 1000
```

Question 8 — Quatorze volumes anonymes

Réponse. Ils viennent d'images qui déclarent `VOLUME` (postgres, mysql, redis, jenkins…) lancées **sans** `-v` : chaque `run` crée un volume anonyme, et `podman rm` ne le supprime pas. Pour chacun : `podman ps -a --filter volume=<nom>` dit quel conteneur l'utilise (aucun → orphelin), et `podman volume inspect` donne la date de création. Pour éviter : nommer les volumes, et supprimer les conteneurs avec `rm -v`. `podman volume prune` supprime **tous** les volumes non utilisés par un conteneur existant — y compris un `pgdata` nommé dont le conteneur a été recréé la veille.

Pourquoi. Le moteur ne sait pas distinguer « temporaire » de « précieux » ; il ne connaît que « monté par un conteneur existant ». Un volume nommé dont le conteneur a été supprimé est, pour `prune`, un déchet.

Nuance. `podman volume ls --filter dangling=true` liste les volumes montés par aucun conteneur existant, sans rien supprimer — mais un `pgdata` nommé dont le conteneur vient d'être

supprimé y figure aussi. C'est une liste à **lire**, pas à passer telle quelle à `rm`.

Exemple.

```
podman image inspect postgres:16-alpine --format '{{.Config.Volumes}}'   # map[/var/lib/postgresql/data:{}]
podman volume ls --filter dangling=true
podman ps -a --filter volume=pgdata --format '{{.Names}}'
```

Question 9 — `tar` à chaud

Réponse. Pendant que PostgreSQL écrit, les fichiers de données et le journal (WAL) changent en permanence ; `tar` les lit l'un après l'autre, à des instants différents. L'archive est un mélange d'états : au mieux PostgreSQL rejoue le WAL et s'en sort, au pire il refuse de démarrer (`invalid checkpoint record`). Les deux méthodes correctes : **sauvegarde logique** (`pg_dump`, cohérente à chaud, portable), ou **sauvegarde physique conteneur arrêté** (`volume export`, `tar`). Pour le quotidien : `pg_dump`, qui n'exige aucune interruption.

Pourquoi. `pg_dump` ouvre une transaction et lit un instantané cohérent de la base. Une copie de fichiers n'a pas cette notion.

Nuance. Une sauvegarde jamais restaurée n'existe pas : le test de restauration dans un conteneur jetable fait partie de la procédure. Et les vrais outils de sauvegarde physique à chaud (`pg_basebackup`, pgBackRest) existent, mais dépassent ce labo.

Exemple.

```
podman exec db pg_dump -U postgres > dump.sql          # à chaud, cohérent
podman stop db && podman volume export pgdata -o pgdata.tar && podman start db   # physique, arrêté
```

Question 10 — `volume export` contre `tar` dans un conteneur

Réponse. La seconde commande lance un conteneur Alpine jetable avec **deux** montages : le volume `pgdata` sur `/data` en lecture seule, et votre dossier courant sur `/backup` ; `tar` y archive `/data` vers `/backup/pgdata.tgz`, qui apparaît donc chez vous. Elle a besoin d'un conteneur parce que, vu de l'hôte, les fichiers du volume appartiennent à l'UID 100069 et que votre `tar` ne peut pas les lire — dans le conteneur, ils sont lisibles. Cette méthode fonctionne **aussi avec Docker** ; `podman volume export` est propre à Podman.

Pourquoi. Un conteneur utilitaire est le moyen universel de « voir un volume avec les bons droits ». `podman volume export` fait la même chose en interne, sans conteneur.

Nuance. `podman unshare tar czf … $(podman volume inspect pgdata --format '{{.Mountpoint}}')` est une troisième voie, Podman uniquement. Dans tous les cas : conteneur arrêté.

Exemple.

```
podman run --rm -v pgdata:/data:ro -v ./:/backup alpine tar czf /backup/pgdata.tgz -C /data .
tar tzf pgdata.tgz | head -3
```

Question 11 — Secret en tmpfs

Réponse. La couche writable est **sur le disque** de l'hôte (`~/.local/share/containers/storage/overlay/…`) : un secret y laisse une trace tant que le conteneur existe, et potentiellement dans un `podman commit`, un `podman export`, ou un instantané de disque. Un tmpfs est **en mémoire** : jamais écrit sur le disque, effacé à l'arrêt, inaccessible via le stockage du moteur. Contrepartie : sa taille est prise sur la RAM et comptée dans `--memory`.

Pourquoi. tmpfs est un système de fichiers du noyau dont le support est la mémoire (et le swap, si activé — d'où l'intérêt de limiter `size=`).

Nuance. Un tmpfs ne protège pas contre un `podman exec` pendant que le conteneur tourne : il protège contre la **persistance**, pas contre l'accès en direct.

Exemple.

```
podman run --rm --tmpfs /run/secrets:size=1m alpine sh -c 'echo jeton > /run/secrets/t; df -h /run/secrets'
```

Question 12 — Permission denied sur RHEL

Réponse. RHEL utilise **SELinux** en mode *enforcing* : les fichiers de `./conf` portent l'étiquette de votre dossier personnel (`user_home_t`), que les processus conteneurisés n'ont pas le droit de lire, root ou pas. Correction : `-v ./conf:/etc/app:ro,Z`, qui ré-étiquette le répertoire en `container_file_t`. Sur Ubuntu/WSL, SELinux est absent (AppArmor), l'option est acceptée et ignorée : l'écrire ne coûte rien et rend la commande portable.

Pourquoi. SELinux ajoute un contrôle d'accès **obligatoire**, indépendant des UID et des `chmod`. `--user 0` ne change rien : la décision se prend sur les étiquettes.

Nuance. `:Z` étiquette le répertoire pour **ce conteneur** ; `:z` pour tous. Ne jamais appliquer `:Z` à un répertoire système (`/home`, `/etc`) : le ré-étiquetage casse l'accès pour le reste du système.

Exemple.

```
podman run --rm -v ./conf:/etc/app:ro,Z alpine cat /etc/app/app.yaml   # RHEL : OK ; WSL : OK aussi
ls -Z conf   # RHEL : …container_file_t… ; WSL : ? (pas d'étiquette)
```

Question 13 — Deux API, un volume `uploads`

Réponse. Chaque conteneur monte **son** volume local : un document envoyé au réplica 1 est invisible du réplica 2, et le téléchargement échoue une fois sur deux. Si les deux tournent sur des machines différentes, le volume n'est même pas partageable. En entreprise, les fichiers utilisateurs vont dans un **stockage objet** (S3, MinIO, Azure Blob) accessible par HTTP depuis n'importe quel réplica ; l'application ne garde **aucun état local**. C'est la condition pour qu'un conteneur soit jetable : on peut en tuer un, en lancer trois, les déplacer — rien n'est perdu.

Pourquoi. Un volume est local à un hôte et à un moteur. L'état partagé doit être dans un service conçu pour ça (base, stockage objet, cache réseau).

Nuance. Pour un seul réplica sur une seule machine, le volume fonctionne — c'est ce qui rend l'erreur invisible jusqu'au jour où l'on scale. Kubernetes a des volumes partagés (`ReadWriteMany`, NFS), mais ils restent l'exception.

Exemple.

```
podman run -d --name api1 -v uploads1:/app/uploads … ; podman run -d --name api2 -v
uploads2:/app/uploads …
podman exec api1 touch /app/uploads/doc.pdf ; podman exec api2 ls /app/uploads    # vide
```